

Projet de soutenance

"Wystrelia's Bot"



Gwenael SAMYN

Jérôme BERTIN

Jules BEAUFORT

Sommaire

Cadre Fonctionnel.....	3
Problématique :.....	3
Description du projet :.....	3
Spécifications fonctionnelles :.....	4
1 - Bot Discord.....	4
A — Pôle Modération (Version 1) : la sécurité du serveur.....	4
B — Pôle Communauté (Version 2) : l'animation et l'engagement.....	5
2 - Interface Web.....	6
User stories :.....	7
Contraintes et livrables attendus :.....	8
Cadre Organisationnel.....	9
Méthode Agile :.....	9
1 - La méthodologie Scrum.....	9
A — Product Backlog.....	10
B — Sprint Backlog.....	11
2 - Kanban.....	12
Cadre Technique.....	13
Outils et environnements :.....	13
Acteurs :.....	13
Règles métiers :.....	14
Diagramme de cas d'utilisation :.....	17
Architecture du projet.....	18
Structure du projet.....	18
Maquettes.....	18

Cadre Fonctionnel

Problématique :

Discord est une plateforme de communication qui permet aux utilisateurs d'interagir par texte, voix et vidéo. Chacun peut rejoindre ou créer ses propres communautés, organisées autour de sujets, d'intérêts ou de jeux variés.

Certaines de ces communautés emploient des « **bots** » : des programmes automatiques qui remplissent différentes fonctions selon leurs objectifs. Mais cela oblige souvent à cumuler une multitude de bots, dont les fonctionnalités les plus utiles sont fréquemment payantes – voire qui, à mesure qu'ils gagnent en notoriété, deviennent entièrement payants.

C'est de ce constat qu'est née l'idée de créer notre propre bot : un outil privé et unique, conçu sur mesure pour répondre aux besoins de la communauté comme à nos propres envies.

Description du projet :

Le projet est un **écosystème de gestion de communauté en ligne**, composé de deux outils complémentaires :

- **Un bot Discord** : un programme automatique qui travaille en coulisses sur le serveur. Il surveille et agit en continu, sans intervention humaine.
- **Un tableau de bord web** : un site qui sert de poste de commande, avec une interface visuelle simple (*boutons, menus*) pour piloter le bot sans taper la moindre commande technique.

L'objectif est triple : **automatiser la modération** (gestion des messages indésirables, du spam...), **animer la communauté** via un système d'engagement (récompenses, niveaux) qui incite les membres à participer, et **centraliser tous les réglages** au même endroit, sans aucune connaissance technique requise.

Le tableau de bord est privé, réservé aux administrateurs, qui s'y connectent directement avec leur compte Discord. Enfin, le bot est conçu sur mesure pour une communauté précise : le serveur « Wystrelia ».

Spécifications fonctionnelles :

1 - Bot Discord

Le bot se construit en deux étapes, qui correspondent à deux grandes missions.

A — Pôle Modération (Version 1) : la sécurité du serveur

Cette première version assure la tranquillité et la sécurité de la communauté. Elle repose sur trois piliers.

D'abord, une **modération automatique** : le bot surveille en permanence les messages et intervient seul, sans qu'un humain ait à le faire. Il bloque les mots interdits définis à l'avance, repère le spam (messages envoyés en masse pour noyer les discussions), et supprime automatiquement les liens ainsi que les invitations vers d'autres serveurs Discord, souvent utilisés à des fins publicitaires ou malveillantes.

Ensuite, une série de **commandes** mises à disposition des modérateurs. Ce sont des raccourcis qu'ils tapent pour agir rapidement sur un membre ou un salon. Elles permettent par exemple de consulter les informations d'un membre, de lui adresser un avertissement (et de consulter ou retirer ses avertissements), de le réduire au silence temporairement, de l'exclure pour une durée définie, de l'expulser ou de le bannir définitivement, de verrouiller un salon pour suspendre les échanges, ou encore d'effacer des messages en bloc.

Enfin, un **journal d'activité** (les « logs ») : chaque action est enregistrée et un compte rendu est automatiquement publié dans un salon réservé à l'équipe de modération. Cela garantit une traçabilité complète et permet à toute l'équipe de suivre ce qui se passe, en toute transparence.

B — Pôle Communauté (Version 2) : l'animation et l'engagement

Cette seconde version vise à rendre l'expérience plus vivante et à fidéliser les membres.

Elle propose des **annonces automatisées** : messages de bienvenue à l'arrivée d'un nouveau membre, souhaits d'anniversaire, et notifications lorsqu'un membre lance une diffusion en direct (un « stream »).

Elle gère aussi l'**attribution automatique de rôles**. Un rôle, sur Discord, est une étiquette qui accorde un statut ou des autorisations. Le bot peut en attribuer un dès qu'une personne rejoint le serveur, et d'autres au fur et à mesure qu'elle progresse.

Cette progression repose sur un **système de niveaux** qui récompense la participation : les membres gagnent de l'expérience en discutant ou en passant du temps dans les salons vocaux, puis grimpent en niveau. Chacun peut consulter sa progression, voir le classement des membres les plus actifs, et les administrateurs disposent d'outils pour ajuster ou réinitialiser ces points.

S'y ajoutent plusieurs fonctions d'animation et d'organisation :

- Des **messages enrichis** (les « embeds ») : des encadrés soignés et mis en forme, plus lisibles qu'un simple texte, que l'on peut créer, configurer, enregistrer et publier.
- Une **réponse personnalisée** lorsqu'on mentionne le bot : il répond alors par un message amusant choisi au hasard.
- Des **salons réservés aux médias** (Media-Only-Channels) : des espaces où l'on ne peut publier que des images ou vidéos, sans discussion, pour garder une galerie propre.
- Un **planning hebdomadaire** que l'on peut créer et modifier directement, pour annoncer les événements de la semaine.

2 - Interface Web

Le second outil de l'écosystème est un **tableau de bord web** (le « dashboard »), pensé comme le poste de commande du bot. Il permet de tout configurer et superviser depuis une interface visuelle, sans jamais toucher au code, ce qui rend le projet utilisable même par des personnes non développeuses.

Une connexion sécurisée. Le tableau de bord est privé. On s'y connecte directement avec son compte Discord, via le système d'authentification officiel de Discord (OAuth2). À la connexion, l'outil vérifie automatiquement que la personne dispose bien des droits d'administrateur sur le serveur Wystrelia ; à défaut, l'accès est refusé. Les réglages sensibles restent ainsi réservés à l'équipe habilitée.

Une vue d'ensemble. Dès l'arrivée, l'administrateur accède à un récapitulatif de la vie du serveur : le nombre de membres, les arrivées et les départs, le volume de messages par jour, ou encore le classement des membres les plus actifs. Cette page donne en un coup d'œil l'état de santé et l'activité de la communauté.

Un éditeur de configuration. C'est le cœur du dashboard : il permet de piloter, sans la moindre commande technique, l'ensemble des fonctionnalités du bot. On y retrouve :

- **L'auto-modération** : ajouter, modifier ou supprimer les mots interdits, et définir les rôles exemptés des filtres.
- **Le système de niveaux** : régler le nombre de niveaux, les paliers et les rôles associés, ainsi que le multiplicateur d'expérience.
- **Les annonces automatisées** : personnaliser le message et l'image de bienvenue, gérer la liste et les messages d'anniversaire, et déclarer les membres streamers à suivre (avec détection des diffusions Twitch).
- **Les rôles automatiques** : choisir les rôles attribués à l'arrivée d'un nouveau membre.
- **Les messages enrichis (« embeds »)** : créer, modifier, enregistrer puis publier ces encadrés mis en forme.
- **Les salons réservés aux médias (« MOC »)** : désigner les salons soumis à cette règle.

Une gestion des membres. Enfin, le tableau de bord affiche la liste complète des membres avec leurs informations utiles (avatar, pseudo, date d'arrivée, niveau, expérience, nombre d'avertissements) et permet de déclencher certaines actions de modération en un clic, sans avoir à taper de commande sur Discord.

User stories :

Les user stories (récits utilisateurs) sont de courtes descriptions d'une fonctionnalité du point de vue de l'utilisateur, suivant le format : « En tant que [utilisateur], je veux [action] afin de [objectif]. » Elles décrivent un besoin de façon simple et centrée sur la valeur apportée.

US-08 — Se connecter au dashboard Priorité haute

En tant que administrateur,

Je veux me connecter via OAuth2 Discord,

Afin d'accéder au dashboard en toute sécurité.

Critères d'acceptation :

- **Étant donné** un utilisateur qui tente de se connecter,
- **Quand** il s'authentifie via Discord,
- **Alors** l'accès est refusé s'il n'est pas administrateur du serveur sinon il accède au dashboard.

US-24 — Médias-Only-Channels (MOC) Priorité moyenne

En tant que administrateur,

Je veux définir des salons réservés aux médias,

Afin de faire respecter un canal images/vidéos uniquement.

Critères d'acceptation :

- **Étant donné** un salon configuré en MOC,
- **Quand** un membre y envoie un message sans média,
- **Alors** le message est supprimé.

US-26 — Réponse aléatoire au ping du bot Priorité Basse

En tant que membre,

Je veux que le bot réponde de façon personnalisée quand on le mentionne,

Afin de rendre l'expérience plus vivante.

Critères d'acceptation :

- **Étant donné** un message mentionnant le bot,
- **Quand** il reçoit un ping,
- **Alors** il répond avec un message aléatoire issu de la banque configurée.

Contraintes et livrables attendus :

Le projet s'inscrit dans un cadre défini à l'avance. Certains choix nous ont été **imposés** (les contraintes), et un ensemble de productions est **attendu** à la livraison (les livrables).

Les contraintes

- **Technique** : la solution doit reposer sur une pile technologique imposée — *React* avec *TypeScript* pour l'interface web (la partie visible, côté navigateur), *NestJS* pour le serveur (la partie qui traite les données en coulisses), *TypeORM* pour faire le lien avec la base de données, et *MySQL* comme base de données (l'endroit où sont stockées toutes les informations).
- **Sécurité** : l'accès au tableau de bord doit être authentifié et restreint aux administrateurs, et toutes les données saisies doivent être contrôlées avant traitement. La sécurité n'est pas une option, mais une exigence.
- **Qualité** : l'application doit être accompagnée de tests automatisés (des vérifications qui s'assurent que le code fonctionne comme prévu) à plusieurs niveaux — unitaires, d'intégration et fonctionnels — et respecter des règles d'accessibilité.
- **Méthodologique** : le projet est mené à trois, en méthode Agile/Scrum, avec une organisation en sprints.
- **Fonctionnelle** : l'outil est privé et conçu sur mesure pour un seul serveur, « *Wystrelia* », et non pour une diffusion publique.

Les livrables attendus

- L'**application fonctionnelle** — le bot Discord et le tableau de bord web — déployée en environnement de production (c'est-à-dire mise en ligne et réellement utilisable).
- Le **code source** versionné et hébergé (sur GitHub), accompagné de sa documentation.
- La **base de données** et sa **stratégie de sauvegarde**, pour garantir qu'aucune donnée n'est perdue en cas d'incident.
- La **chaîne d'intégration et de déploiement continu** (CI/CD) : un système qui automatise les tests et la mise en ligne à chaque modification, pour livrer plus vite et plus sûrement.
- Le **dossier de projet** et son support de présentation, remis en vue de la soutenance.

Cadre Organisationnel

Méthode Agile :

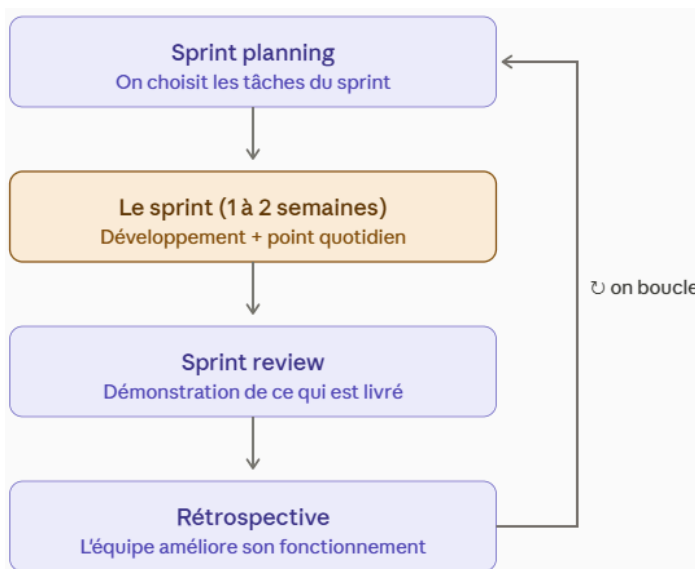
La méthode Agile est une philosophie de gestion de projet qui privilégie la souplesse et l'adaptation plutôt qu'un plan rigide fixé à l'avance.

Plutôt que de tout définir au départ puis de livrer le projet en une seule fois, on avance par petites étapes successives. À chaque étape, une partie fonctionnelle du produit est livrée, ce qui permet de recueillir des retours rapidement et d'ajuster la suite en fonction des besoins réels.

Agile repose sur quatre valeurs clés (issues du *Manifeste Agile*, 2001) :

- Privilégier les individus et les échanges
- Un produit qui fonctionne
- La collaboration
- La capacité à s'adapter au changement.

1 - La méthodologie Scrum



Scrum est un cadre de travail concret qui applique la philosophie Agile. Il organise le projet en *sprints* : des cycles de travail courts et réguliers (généralement une à deux semaines) au terme desquels une partie fonctionnelle du produit doit être livrée.

Au début de chaque sprint, on sélectionne un ensemble de tâches prioritaires à réaliser ; à la fin, on présente le résultat puis on identifie les améliorations possibles pour le sprint suivant.

Scrum s'appuie sur des rôles définis (Product Owner, Scrum Master, équipe de développement) et sur des réunions régulières, dont un court point quotidien qui permet de suivre l'avancement et de lever rapidement les blocages.

A — Product Backlog

Le product backlog comprend l'ensemble des fonctionnalités du projet (user stories), organisé par ordre de priorité, pour les sprints.

Tt User Story	Tt Description	☰ Priorité
US-01	Filtrer les mots interdits	Priorité Haute ▾
US-02	Avertir un membre	Priorité Haute ▾
US-03	Bannir un membre	Priorité Haute ▾
US-04	Bloquer les spams	Priorité Haute ▾
US-05	Gagner de l'XP	Priorité Basse ▾
US-06	Consulter mon rang	Priorité Basse ▾
US-07	Accueillir un nouveau membre	Priorité Basse ▾
US-08	Se connecter au dashboard	Priorité Haute ▾
US-09	Suivre l'activité du serveur	Priorité Basse ▾
US-10	Modérer en un clic	Priorité Moyenne ▾
US-11	Consulter les informations d'un membre	Priorité Moyenne ▾
US-12	Gérer les avertissements	Priorité Moyenne ▾
US-13	Appliquer un timeout Discord	Priorité Moyenne ▾
US-14	Verrouiller et déverrouiller un salon	Priorité Moyenne ▾
US-15	Effacer des messages en bloc	Priorité Moyenne ▾
US-16	Vérifier la disponibilité du bot	Priorité Basse ▾
US-17	Afficher l'aide de modération	Priorité Basse ▾
US-18	Filtrer les liens et invitations	Priorité Haute ▾
US-19	Exemption de filtres	Priorité Moyenne ▾
US-20	Gérer le système de niveaux	Priorité Basse ▾
US-21	Classement des membres actifs	Priorité Basse ▾
US-22	Ajuster l'XP et le niveau	Priorité Basse ▾
US-23	Réinitialiser les points	Priorité Basse ▾

Tt User Story	Tt Description	🔍 Priorité
US-24	Média-Only Channels (MOC)	Priorité Moyenne ▾
US-25	Envoyer un embed enregistré	Priorité Moyenne ▾
US-26	Réponse aléatoire au ping du bot	Priorité Basse ▾
US-27	Annoncer les anniversaires	Priorité Basse ▾
US-28	Suivi des streams Twitch	Priorité Basse ▾
US-29	Afficher l'historique des actions	Priorité Haute ▾
US-30	Afficher un compteur de membres	Priorité Basse ▾

B — Sprint Backlog

Sprint 1 : Fondations, Sécurité et Logs

[A COMPLETER]

Sprint 2 : Automatisation et Modération Avancée

[A COMPLETER]

Sprint 3 : Gamification et Animations

[A COMPLETER]

2 - Kanban

Kanban est une méthode de visualisation du travail. Elle repose sur un tableau divisé en colonnes représentant les étapes d'avancement d'une tâche — par exemple « À faire », « En cours » et « Terminé ». Chaque tâche est matérialisée par une carte que l'on déplace de colonne en colonne au fur et à mesure de sa progression.

L'objectif est de voir en un coup d'œil l'état d'avancement du projet, d'identifier les blocages et de limiter le nombre de tâches traitées simultanément pour rester efficace.

Pour respecter cette méthode nous avons organisé un tableau Trello, et transféré notre product backlog priorisé sur celui-ci.



Cadre Technique

Outils et environnements :

GitHub, GitHub Actions, Visual Studio Code, Discord, NestJS, React, Drive, MySQL2, TypeORM, Figma, TailwindCSS

Acteurs :

- **Administrateur** : accède au dashboard, configure le bot, modère via l'interface.
- **Modérateur** : utilise les commandes Discord pour sanctionner et gérer les membres.
- **Membre** : participe au serveur et bénéficie des fonctionnalités communautaires.
- **Bot** : exécute l'auto-modération, les annonces et les actions définies.

Règles métiers :

Les **règles métier** (ou règles de gestion) sont les lois propres au fonctionnement du projet : elles définissent ce que l'application doit autoriser, interdire ou imposer, indépendamment de la manière dont elles sont programmées.

Là où une fonctionnalité décrit *ce que fait* l'outil, la règle métier précise *selon quelles conditions* : elle fixe la mécanique exacte et non négociable que le bot et le tableau de bord doivent toujours respecter.

Les règles suivantes constituent ainsi le socle logique du projet, sur lequel s'appuient aussi bien les fonctionnalités que la base de données.

Accès & rôles

- **RG-01** — Seul un rôle doté de la permission Administrateur Discord donne accès au dashboard ; aucun autre rôle ne la possède. La qualité d'administrateur est revérifiée à chaque connexion (OAuth2 Discord).
- **RG-02** — Seul un administrateur peut consulter et modifier la configuration du bot.
- **RG-03** — Les sanctions respectent la hiérarchie Administrateur > Modérateur > Membre : un membre ne peut sanctionner un rôle de rang égal ou supérieur.

Modération automatique

- **RG-04** — La liste des mots interdits est globale au serveur.
- **RG-05** — Tout message contenant un mot interdit est supprimé automatiquement, sauf si l'auteur porte un rôle exempté (modération + rôles « amis » configurables).
- **RG-06** — Par défaut tous les liens sont bloqués ; le droit d'en publier s'obtient en atteignant, par la montée en niveau, le rôle d'exemption.
- **RG-07** — Les invitations vers d'autres serveurs sont bloquées en permanence, sauf modération et rôles « amis ».
- **RG-08** — Le spam est détecté au-delà de 5 messages en 5 secondes, ou de 5 mentions dans un même message.
- **RG-09** — Face au spam, le bot supprime les messages, applique un mute automatique et journalise l'action ; il n'attribue jamais d'avertissement de lui-même.

Sanctions

- **RG-10** — Seuls modérateurs et administrateurs peuvent exécuter les commandes de sanction.
- **RG-11** — Les avertissements sont créés et retirés manuellement ; il n'y a pas d'escalade automatique.
- **RG-12** — Les avertissements sont conservés indéfiniment (traçabilité) mais peuvent être supprimés manuellement ; chaque création ou suppression est journalisée.
- **RG-13** — Le mute automatique anti-spam a une durée fixe configurable (défaut 1 h), distincte du timeout manuel dont la durée est fixée par le modérateur.

Journalisation

- **RG-14** — Toute action de modération — commande du bot ou action Discord native, automatique ou manuelle — génère un log unique (type, cible, auteur, motif, horodatage), consultable sur le dashboard et publié dans le salon de modération du type d'action concerné ainsi que dans le salon `all_logs`.

Système de niveaux

- **RG-15** — L'XP est unifiée : messages et activité vocale alimentent une seule barre de progression par membre (un seul niveau, un seul jeu de paliers), afin que les rôles liés aux paliers restent accessibles quel que soit le mode de participation.
- **RG-16** — Un membre gagne de l'XP en publiant des messages, dans la limite d'un gain par cooldown (défaut 60 s).
- **RG-17** — Un membre gagne de l'XP par minute de présence en vocal.
- **RG-18** — Sont configurables depuis le dashboard : XP par message, XP par minute de vocal, cooldown, niveau maximum, paliers et rôles associés, et multiplicateur global.
- **RG-19** — Le passage d'un palier attribue le rôle associé et retire celui du palier précédent (rôles remplacés, non cumulés).

Salons médias (MOC)

- **RG-20** — Chaque salon MOC déclare ses types de contenu autorisés (images, vidéos, fichiers, liens avec domaines autorisés, texte d'accompagnement). Tout message non conforme — type interdit, ou texte seul quand il n'est pas autorisé — est supprimé.

Accueil, animations & communauté

- **RG-21** — À l'arrivée d'un membre, un ou plusieurs rôles de bienvenue (configurables) lui sont attribués automatiquement.
- **RG-22** — Le compteur ne comptabilise que les membres humains (hors bots) ; le nom du salon vocal est rafraîchi périodiquement (5-10 min) pour respecter les limites de l'API Discord.
- **RG-23** — Les anniversaires sont saisis dans le dashboard (réservé modération + amis) sous forme d'ID Discord + date ; un souhait est publié automatiquement le jour J à 10 h (Europe/Paris).
- **RG-24** — Pour chaque chaîne Twitch déclarée (par nom d'utilisateur), le bot détecte le passage en direct et publie une annonce avec les informations du stream dans un salon dédié.

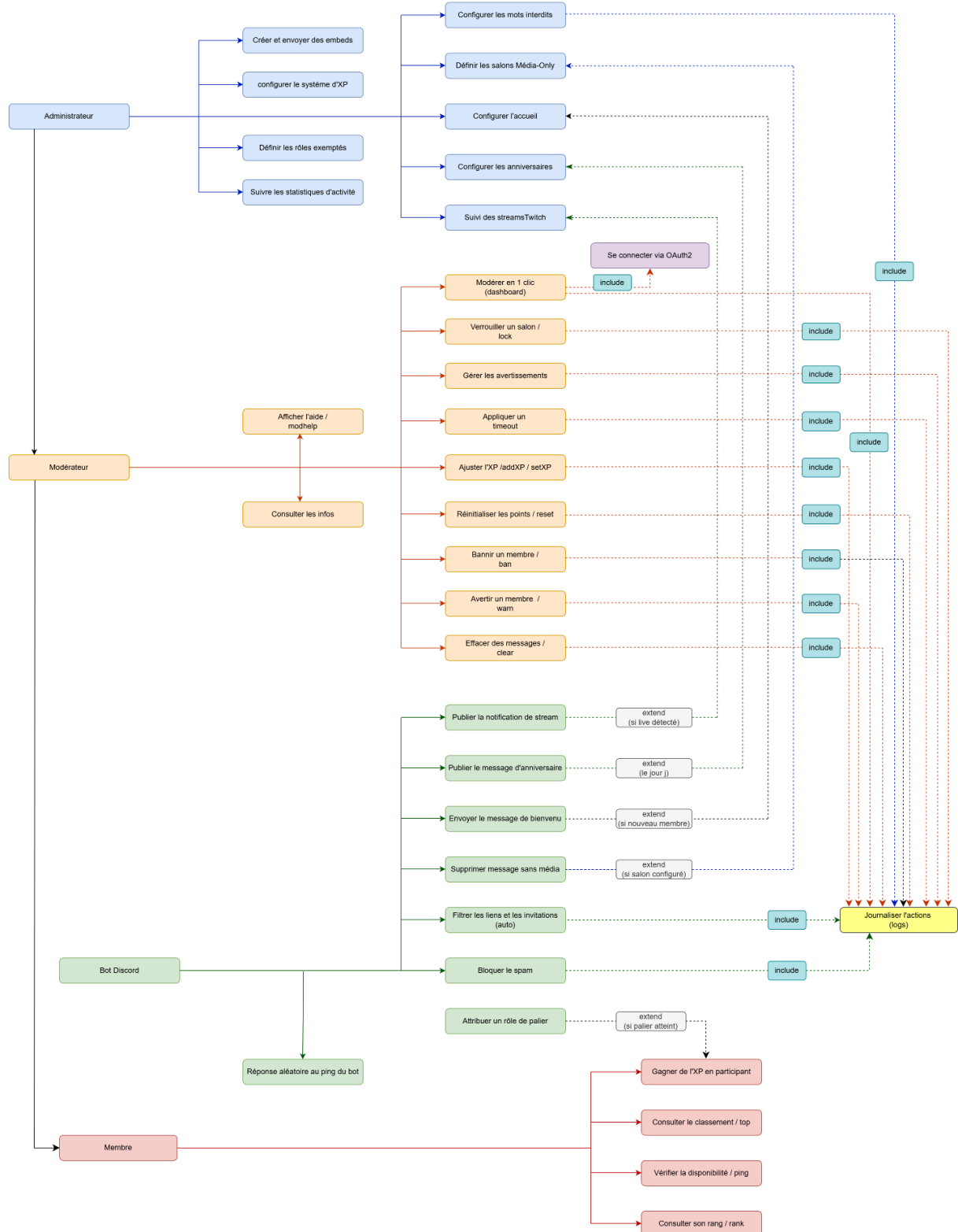
Données & cycle de vie

- **RG-25** — Les données d'un membre (XP, niveau, avertissements, logs) sont rattachées à son identifiant Discord et conservées même après son départ du serveur ; un retour ne réinitialise pas son historique.

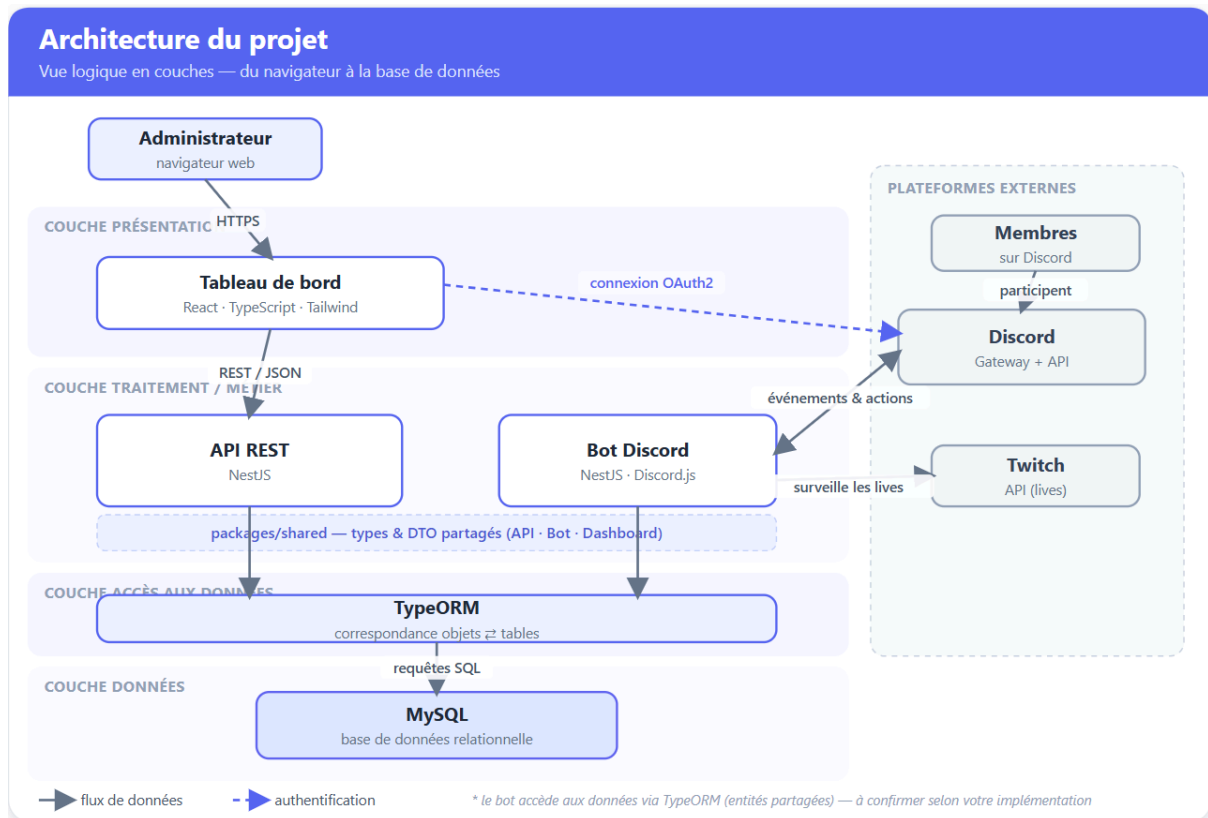
Animation — embeds

- **RG-26** — Les embeds sont composés et enregistrés depuis le dashboard, puis réutilisables et publiables à la demande par un administrateur.

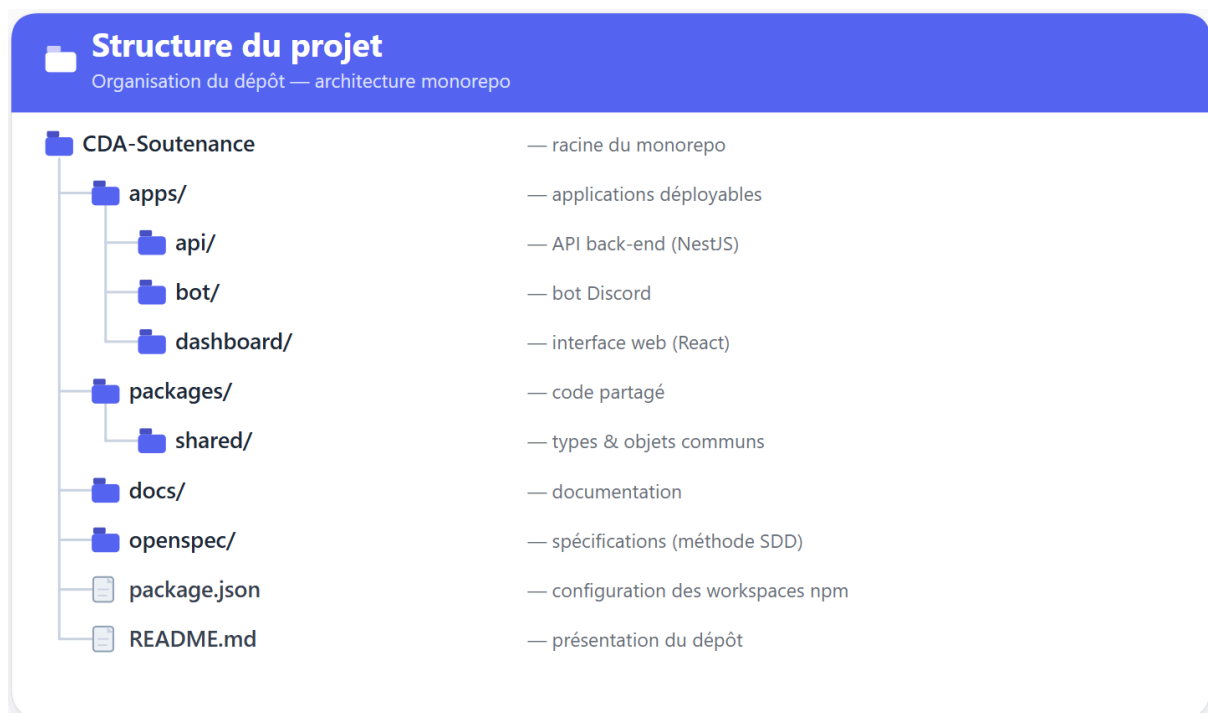
Diagramme de cas d'utilisation :



Architecture du projet :



Structure du projet :



Maquettes :

x